عملیات	Array Collection	LinkedList Collection	دلیل تفاوت
insertOne (آخر)	**O(1)**	**O(1)**	آرایه push_back :معمولاً O(1) است گاهی resize می‌شود. لیست: چون tailداریم ، مستقیم به آخر وصل می‌کنیم.
deleteOne (ابتدا)	**O(n)**	**O(1)** *(گر آی‌دی head باشد)*	آرایه erase(begin+i) :باعث **شیفت** کل عناصر بعدی می‌شود O(n).لیست: اگر node هدف head باشد، فقط head = head->next و تنظیم prev => O(1).
deleteOne (وسط)	**O(n)**	**O(n)**	آرایه: پیدا کردن + شیفت دادن O(n). لیست: برای رسیدن به نود وسط باید پیمایش کنیم O(n) ، ولی حذف خودش با تغییر پوینترها O(1)است.
findByID	**O(n)**	**O(n)**	در هر دو، جستجو خطی است (حلقه/پیمایش) از ابتدا تا پیدا شدن (id).
count	**O(1)**	**O(n)**	آرایه data.size() :مستقیم O(1). لیست: در کد count()هر بار کل لیست را می‌شمارد.O(n)

سوالات مفهومی:

۱. چرا در لیست پیوندی DeleteOne() زمان بر است؟

زیرا در لیست پیوندی، نودها در حافظه پشت‌سرهم نیستند و هیچ دسترسی مستقیمی به یک نود خاص وجود ندارد پس باید برای حذف یک عنصر با id مشخصش:

باید از ابتدای لیست (head) شروع کنیم.تک‌تک نودها را بررسی کنیم تا نودی با idموردنظر پیدا شود. علاوه بر خود نود، باید به نود قبلی هم دسترسی داشته باشیم تا اشاره‌گرها را اصلاح کنیم.

البته اگر با id کار نداشتیم و صرفا میخواستیم نود سر لیست را حذف کنیم پیچیدگی زمان برابر با O(1) می‌شد.

۲. چرا جدا بودن Parserاز Storageمفید است؟

در این حالت parser صرفا دستور را میفهمد و storage و engine وظیفه ی نگهداری و انجام عملیات را بر عهده دارد و از مزیت های این دسته بندی میتوان به:

- افزودن دستور جدید بدون دست زدن به Storage
- امکان تغییر نوع ذخیره‌سازی بدون تغییر دادن Parser: (ArrayCollectionرا با LinkedListCollectionجایگزین کنیم)
- Parser و Storage را میتوان جدا جدا تست کرد و کد نویسی تمیز تری خواهیم داشت.